

Taller 1

Taller 1

Uso de Python

Para utilizar Python en este formato es necesario incorporar el siguiente código:

```
# En caso de que no esté instalado el paquete, instalarlo  
# install.packages("reticulate")  
require(reticulate)
```

```
## Loading required package: reticulate
```

Luego de incorporar el paquete, es necesario instalar las librerías de python deseadas:

```
# El comando py_install de reticulate, instala la librería pasada como argumento  
py_install("numpy")
```

```
## Using virtual environment '/cloud/project/r-reticulate' ...
```

```
# Si ya está instalada, comentar este código para que no instale siempre la librería,  
# demorando la compilación
```

Vemos que ahora importa la librería deseada:

```
import numpy as np  
np.arange(5)
```

```
## array([0, 1, 2, 3, 4])
```

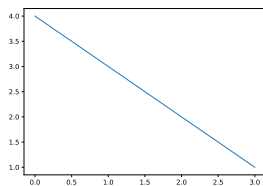
Para gráficos en Python, se pueden ajustar los tamaños de las figuras usando `out.width` y `out.height` como argumentos del fragmento de código. Puede tomar como argumentos porcentajes de las dimensiones del documento, pulgadas o centímetros:

```
py_install("matplotlib")
```

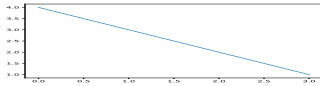
```
## Using virtual environment '/cloud/project/r-reticulate' ...
```

```
## + /cloud/project/r-reticulate/bin/python -m pip install --upgrade --no-user matplotlib
```

```
import matplotlib.pyplot as plt  
plt.plot(np.arange(1,5)[::-1]) # out.width="25%"
```



```
plt.plot(np.arange(1,5)[::-1]) # out.height="0.5in",out.width="2in"
```



Vectores

1. Considerar los siguientes vectores:

- $\vec{u} = (4, -2)$
- $\vec{v} = (2, 1)$
 - a. Calcular la proyección de $\vec{u}$ sobre $\vec{v}$
 - b. Calcular la proyección de $\vec{v}$ sobre $\vec{u}$

Matrices

2. Considerar la siguiente matriz:

- $A = \begin{pmatrix} 185 & -80 \\ -80 & 65 \end{pmatrix}$
 - a. Calcular los autovalores y autovectores
 - b. Expresar A en su forma diagonalizada ($A = V \times D \times V^{-1}$)
 - c. Dividir cada columna de V por su norma, obteniendo la matriz V_o . Comprobar que $A = V_o \times D \times V_o^T$.
¿Cómo haría para invertir la matriz A ?

3. Considerar la matriz $B = \begin{pmatrix} 10 & -7 & 6 \\ -5 & 6 & 2 \end{pmatrix}$

- a. Calcular los autovalores y autovectores de la matriz $B \times B^T$. Obtener una matriz U disponiendo los autovectores como columnas y dividiendo por su norma.
- b. Calcular los autovalores y autovectores de la matriz $B^T \times B$ (Ayuda: los autovalores son los mismos que los de $B \times B^T$, al que se le agrega un cero). Obtener una matriz V disponiendo los autovectores como columnas y dividiendo por su norma.
- c. Descomponer la matriz B según sus valores singulares ($B = U \times D \times V^T$)

Vectores aleatorios

4. Considerar los siguientes datos:

x	y
5820	5880
6030	5970
5850	6060
6120	5940
6240	6000
6000	6240

- a. Calcular el vector de medias, la matriz de covarianza y la matriz de correlación para estos datos.
- b. Calcular la distancia de Mahalanobis y la densidad normal multivariada de cada dato utilizando el vector de medias $\mu = (6000, 6000)$ y matriz de covarianza $\Sigma = \begin{pmatrix} 185 & -80 \\ -80 & 65 \end{pmatrix}$.

Bases de datos

5. Considerar las bases `df_a.csv`, `df_b.csv` y `df_c.csv`.

a. Hacer un scatterplot de ambas variables en cada base. Completar el siguiente código para evitar reescribir instrucciones similares:

```
def scatGG(Base,x_Ch, y_Ch):  
    fig, ax = plt.subplots()  
    sns.scatterplot(data=___, x=___, y=___, alpha=0.5)  
    return fig,ax
```

b. En cada base, calcular el vector de medias, la matriz de covarianza y la matriz de correlación.

c. Hacer gráficos de densidad conjunta de ambas variables, utilizando contornos y heatmaps. Completar el siguiente código para evitar reescribir instrucciones similares:

```
def dCont2DGG(Base,x_Ch, y_Ch):  
    fig, ax = plt.subplots()  
    sns.kdeplot(data=___, x=x_Ch, y=___, cmap="viridis",levels=___)  
    return fig,ax  
  
def dFill2DGG(Base, x_Ch, y_Ch):  
    fig, ax = plt.subplots()  
    sns.kdeplot(data=Base, x=x_Ch, y=y_Ch, cmap="viridis", fill=___, thresh=___)  
    return fig,ax
```

d. Hacer histogramas y qq-plots de cada variable, identificar cuáles de las variables tienen distribución parecida a una normal. Completar el siguiente código para evitar reescribir instrucciones similares:

```
def GG_qqNorm(Base, x_Ch):  
    fig, ax = plt.subplots()  
    stats.probplot(Base[___], dist=___, plot=ax)  
    ax.get_lines()[0].set_markerfacecolor('blue')  
    ax.get_lines()[0].set_alpha(0.5)  
    ax.get_lines()[1].set_color('red')  
    ax.get_lines()[1].set_linestyle('--')  
    return fig,ax  
  
# 7. Histograma con curva Normal superpuesta  
def histGG_Norm(Base, x_Ch):  
    fig, ax = plt.subplots()  
  
    mx, sx = Base[___].____(), Base[___].____()  
  
    sns.histplot(data=Base, x=x_Ch, stat=___, edgecolor="blue", color="white")  
  
    xs = np.linspace(mx - 2.5 * sx, mx + 2.5 * sx, 500)  
    ds = stats.norm.pdf(xs, ___, ___)  
    ax.plot(xs, ds, color="red", linewidth=2)  
  
    return fig,ax
```

e. ¿Cuál de las bases tiene una distribución de sus columnas que se asemeja a una normal multivariada?